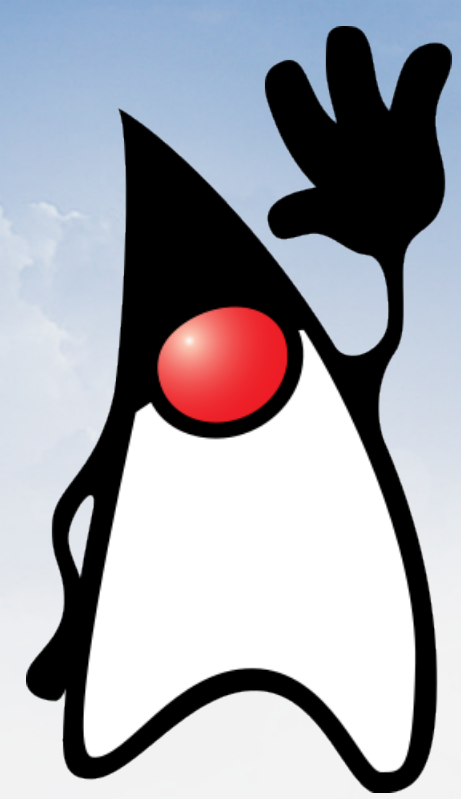




Inheritance part 2

COP 3330
Department of Computer Science
University of Central Florida
Dr. Bacanli

Method Hiding

- Remember method overloading and method overriding?
- Method hiding can be defined as a child class defining some static method with the same signature as the **static** method in the parent class,
 - The method in the child class hides the one in the parent class

instanceof Operator

- Java has a special operator to check to see if a certain object is an instance of some class.
- Returns a boolean value
- Generally using it means “bad design”

instanceof

```
package payments;
public class CardPayment extends Payment{

    public CardPayment(){
        super();
        System.out.println("Card is created");
    }

}
```

```
package payments;
public class Payment{
    private int amount;

    public Payment(){
        amount=400;
        System.out.println("Payment is created");
    }

}
```

```
package payments;
public class PaymentsTester{
    public static void main(String[] args){
        Payment c=new Payment();
        CardPayment cp= new CardPayment();
        System.out.println(cp instanceof Payment);
        System.out.println(cp instanceof CardPayment);
        //System.out.println(cp instanceof cp); INVALID
    }//end of main method
}
```

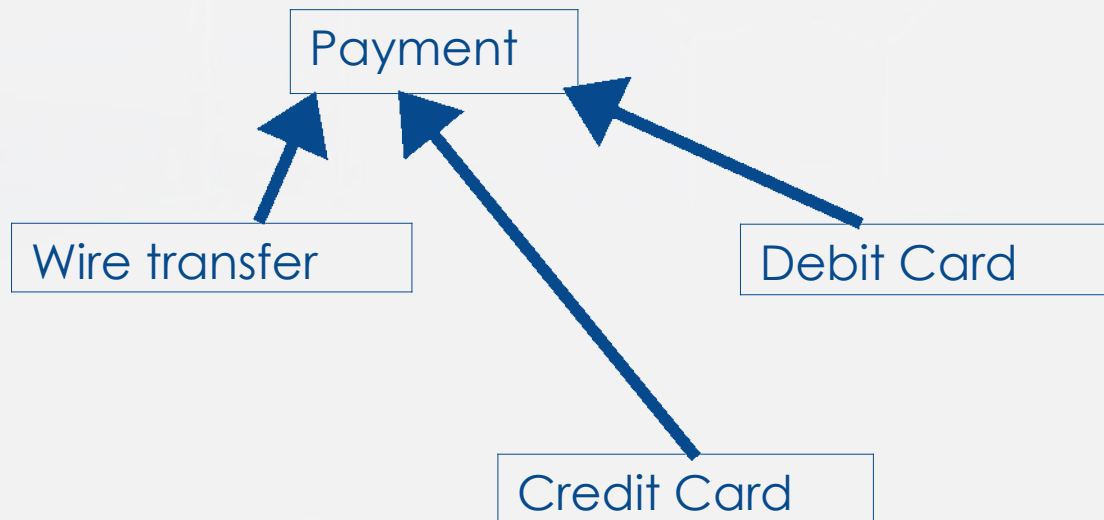
```
Payment is created
Payment is created
Card is created
true
true
```

Multilevel Inheritance

- Just as a family grows, we inherit qualities from our ancestors
- In OOP, the same concept can be applied as we descend the family tree of classes.
- This is known as multilevel inheritance.

Abstract Classes

- If you have created a base class and made multilevel inheritance sometimes the base class becomes abstract
 - Constructor exists! but you cannot create object from that
 - Yes you can extend, in fact made to be extended!



You may not want to use Payment object anywhere. Rather you prefer specific objects

Abstract Class

```
package payments;
public class CardPayment extends Payment{

    public CardPayment(){
        super();
        System.out.println("Card is created");
    }
}
```

```
package payments;
public class CreditCardPayment extends
CardPayment{

    public CreditCardPayment(){
        super();
        System.out.println("CCard is created");
    }
}
```

```
package payments;
public abstract class Payment{
    private int amount;

    public Payment(){
        amount=400;
        System.out.println("Payment is created");
    }

    public void getAmount(){
        return amount;
    }
}
```

```
package payments;
public class PaymentsTester{
    public static void main(String[] args){
        Payment c=new Payment();
        //Compiler Error
    }
}
```

Abstract Class

```
package payments;
public class CardPayment extends Payment{

    public CardPayment(){
        super();
        System.out.println("Card is created");
    }
}
```

```
package payments;
public class CreditCardPayment extends
CardPayment{

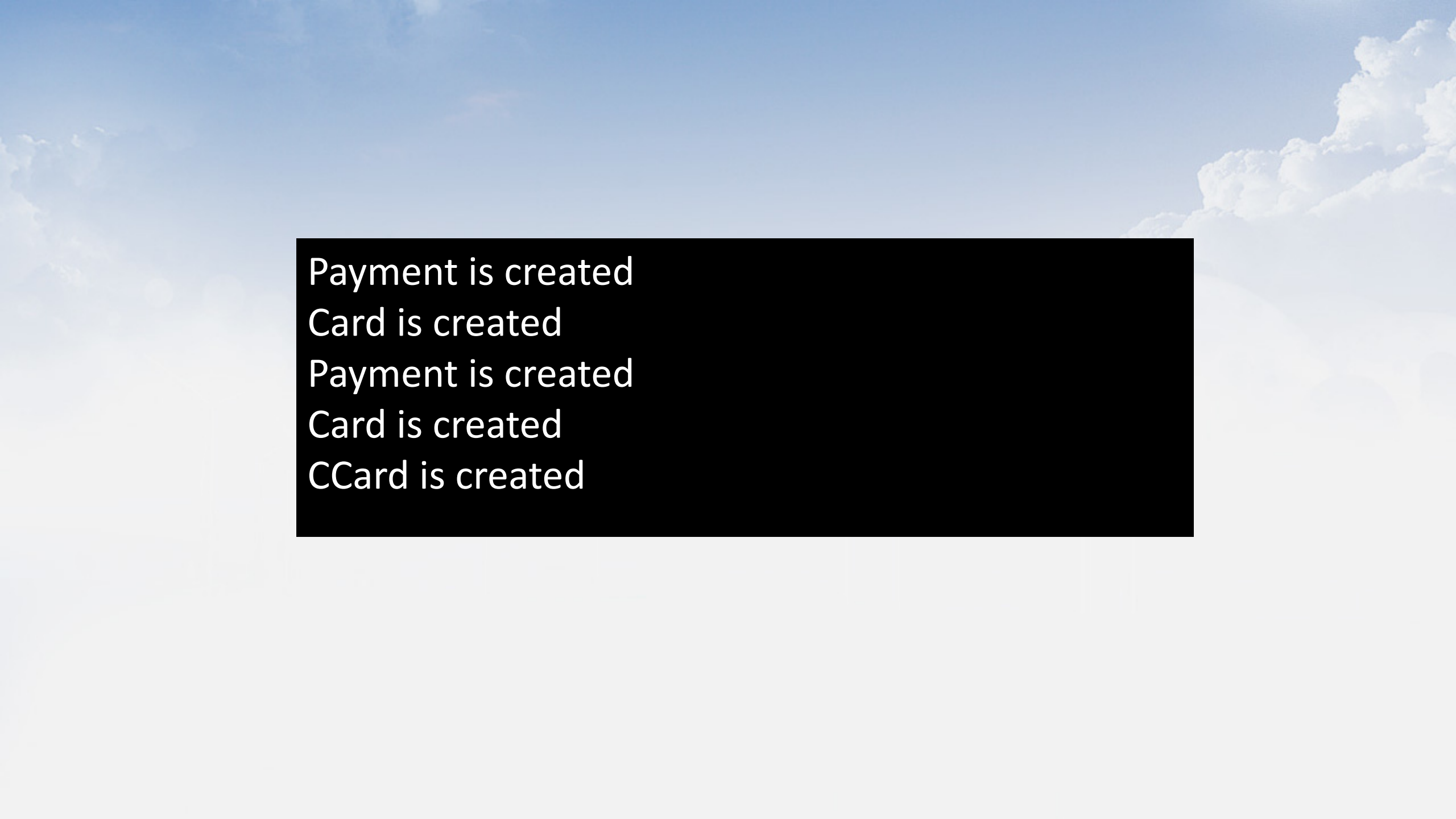
    public CreditCardPayment(){
        super();
        System.out.println("CCard is created");
    }
}
```

```
package payments;
public abstract class Payment{
    private int amount;

    public Payment(){
        amount=400;
        System.out.println("Payment is created");
    }

    public void getAmount(){
        return amount;
    }
}
```

```
package payments;
public class PaymentsTester{
    public static void main(String[] args){
        CardPayment c=new CardPayment();
        CreditCardPayment c1= new
CreditCardPayment();  }
}
```



Payment is created

Card is created

Payment is created

Card is created

CCard is created

Abstract Method

- A method which is made to be overridden/implemented
 - method level of abstract class
- If a class has an abstract method inside
 - It must be an abstract class
- You can still define abstract class without any abstract method inside
 - As we did before

Abstract Method

```
package payments;
public class CardPayment extends Payment{

    public CardPayment(){
        super();
        System.out.println("Card is created");
    }

    @Override
    public void tell(){
        System.out.println("CreditCard is needed");
    }

}
```

Payment is created
Card is created
CreditCard is needed

```
package payments;
public abstract class Payment{
    private int amount;

    public Payment(){
        amount=400;
        System.out.println("Payment is created");
    }

    public abstract void tell();
}
```

```
package payments;
public class PaymentsTester{
    public static void main(String[] args){
        CardPayment c=new CardPayment();
        c.tell();
    }//end of main
}
```

Why do we create abstract methods?

- There is a method
 - Any class extending that class must implement it!
 - Creating a standard
- If you want your class to do that function
 - You must extend the class AND override the abstract method
- abstract method is *kind of* opposite of final method
- ~~public static abstract void tell()~~